Smart Home Support Agent

Midterm AI Agent Blueprint

1. Problem Statement

Despite the widespread use of smart home appliances, troubleshooting them remains a challenging task. People occasionally encounter issues with voice assistants, unresponsive devices, and/or connectivity.

By serving as a knowledgeable first-line support assistant, the Smart Home Support Agent resolves this issue. When a problem is described in natural language, the agent uses a structured knowledge base to find the most semantically similar known problem and provides a clear, practical solution.

Who Gains: Renters and homeowners who use smart home ecosystems (like Alexa and Google) and want quick, dependable self-service assistance without having to deal with complicated paperwork.

2. Option Choice

Option A: One AI Agent

The entire loop is managed by the agent: it will recognize the user's query, use semantic similarity search to reason, and respond by providing a solution. A multi-agent design would add needless overhead.

The deep learning connections are clear and significant because the single-agent design is consistent with the RAG and Transformer-based embedding modules.

3. Agent Architecture

The agent follows a RAG loop: encode ===> retrieve ====> respond. All components are lightweight and run locally in a Colab notebook with a Gradio front-end.

Input: User's natural-language query via Gradio Textbox

Text preprocessing: lowercasing, punctuation removal, and whitespace normalization. clean_text()

Embedding Model: SentenceTransformer (paraphrase-MiniLM-L3-v2) encodes the query into a dense vector.

FAISS Index: IndexFlatIP performs cosine similarity search over pre-encoded issue embeddings.

Knowledge Base: JSON-structured smart home troubleshooting guide.

Output: Top 3 similar issues + resolutions rendered via Gradio Markdown.

Architecture flow:

- User types a query
- clean_text() normalizes the query
- SentenceTransformer encodes it to a 384-dim embedding
- FAISS IndexFlatIP finds the top 3 most similar issue embeddings via cosine similarity
- Matched resolutions are returned and rendered in Gradio

4. Deep Learning Connection

Module 1: Transformers & Attention Mechanisms

Paraphrase-MiniLM-L3-v2, the embedding backbone, is a condensed Transformer encoder. It generates dense vector representations at the sentence level using multi-head self-attention. Every token pays attention to every other token, capturing semantic relationships (for example, "won't connect" = "connection problems") that are completely missed by bag-of-words models.

In the final project, I want to switch to all MiniLM-L6-v2 for better embeddings and maybe think of adding a generative transformer like GPT to create natural-language responses instead of just returning stored resolutions.

Module 2: Representation Learning & Embeddings

The quality of learned representations is the only factor that affects the retrieval quality of the agent. SentenceTransformer can be trained using contrastive learning to cluster semantically similar sentences in embedding space.

5. Agent Framework

Framework: Custom Python pipeline

For this project, I will implement the agent loop manually using:

- Sentence-transformers: embedding model loading and inference
- Faiss-cpu: approximate nearest neighbor index for fast retrieval
- Gradio: an interactive web UI for query input and result display
- spacy/nltk: text preprocessing and tokenization utilities

I will not use heavyweight frameworks like LangChain or CrewAI at this stage (for now). Since the pipeline is a straightforward encode-retrieve-respond loop, which is valuable for debugging

For the final project, I plan to evaluate LangChain's ConversationalRetrievalChain if I add multi-turn memory, for example, like an API search.

6. Tools & Data

Libraries & APIs

- Sentence-transformers: (HuggingFace) pretrained MiniLM embedding model
- Faiss-cpu: Facebook AI Similarity Search for vector retrieval
- Gradio: Rapid UI prototyping for ML demos
- Spacy: NLP preprocessing
- NLTK: sentence tokenization
- PyTorch: tensor operations underlying SentenceTransformer

Knowledge Base / Data

- Scrape or choose 50 to 100 issue-resolution pairs from manufacturer support pages.
- Add device-specific subcategories (lights, locks, thermostats, cameras)
- Explore using a public smart home FAQ dataset if available on Hugging Face datasets

7. Build Plan

Week 1 (Apr 7-13): Finalize blueprint feedback, expand knowledge base with more device categories, and refine text-cleaning pipeline

Week 2 (Apr 14 - 20): Upgrade embedding model (all-MiniLM-L6-v2), add multi-turn conversation memory, and improve query preprocessing

Week 3 (Apr 21-27): Maybe Integrate a better LLM response generation

Week 4 (Apr 28 - Mar 3): Final testing and debugging, polishing the Gradio UI, and writing a document.

8. Anticipated Challenges

Challenge 1: Low Knowledge Base

For queries outside of those categories, FAISS might provide subpar results due to the small corpus. Solution: increase the number of entries and include a confidence threshold. If the top match score is less than 0.6, provide an "I don't know" response instead of an improper/false one.

Challenge 2: Semantic Gap Between Queries and Index

While the index employs structured language (for example, "Wi-Fi connection problems"), users describe issues informally (ex: "my bulb keeps flickering"). Solution: Experiment with different SentenceTransformer models; think about expanding queries using synonym lists or performing a brief LLM paraphrase step prior to retrieval.

Challenge 3: Scaling FAISS to Larger Corpora

IndexFlatIP performs exact searches for 100+ entries, but for future scaling (1000+ entries), IndexFlat will have issues, and I will have to find a stronger indexing unit.

Citations

Hugging Face Model Card:
https://huggingface.co/sentence-transformers/paraphrase-MiniLM-L3-v2

FAISS GitHub (Meta AI Research): https://github.com/facebookresearch/faiss

FAISS Engineering Blog (Meta):
https://engineering.fb.com/2017/03/29/data-infrastructure/faiss-a-library-for-efficient-similarity-search/

Sentence Transformers Pretrained Models Docs:
https://www.sbert.net/docs/sentence_transformer/pretrained_models.html

Hugging Face – all-MiniLM-L6-v2:
https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2